

A Sentinel for Null-Terminated Strings

Document #: P3705R1
Date: 2025-06-16
Project: Programming Language C++
Audience: SG-9 Ranges
LEWG
Reply-to: Eddie Nolan
[<eddiejolan@gmail.com>](mailto:eddiejolan@gmail.com)

Contents

- [1 Motivation](#)
- [2 Design Alternatives](#)
 - [2.1 Sentinel-based design](#)
 - [2.1.1 Exposition-only helper concept *default-initializable-and-equality-comparable-iter-value*](#)
 - [2.1.2 `null\_sentinel` pass-by-value design](#)
 - [2.1.3 `null\_sentinel` pass-by-reference design](#)
 - [2.1.4 `null\_term` CPO](#)
 - [2.2 `unchecked\_take\_before`-based design](#)
 - [2.2.1 exposition-only helper concept *default-initializable-iter-value*](#)
 - [2.2.2 `null\_term` CPO](#)
- [3 History](#)
 - [3.1 Changelog](#)
 - [3.1.1 Changes since P2728R1](#)
 - [3.1.2 Changes since P2728R3](#)
 - [3.1.3 Changes since P2728R4](#)
 - [3.1.4 Changes since P2728R5](#)
 - [3.1.5 Changes since P2728R6](#)
 - [3.1.6 Changes since P2728R7](#)
 - [3.1.7 Changes since P3705R0](#)

- 3.2 Relevant Polls/Minutes
 - 3.2.1 SG9 review of D2728R4 on 2023-06-12 during Varna 2023
 - 3.2.2 SG16 review of P2728R3 on 2023-05-10 (Telecon)

§ 1 Motivation

Consider using `std::views::take` to get the first five characters from a string. If we pass a string literal with five or more characters, it looks okay:

```
static_assert(std::string{std::from_range, "Brubeck" |
std::views::take(5)} == "Brube"); // passes
```

However, if we pass it a string literal with fewer than five characters, we get in trouble:

```
// static_assert(std::string{std::from_range, "Dave" |
std::views::take(5)} == "Dave"); // fails
using namespace std::string_view_literals;
static_assert(std::string{std::from_range, "Dave" | std::views::take(5)}
== "Dave\0"sv); // passes
```

The reason the null terminator is included in the output here is because undecayed string literals are arrays of characters, including the null terminator, which the ranges library treats like any other array:

```
#include <algorithm>
#include <array>
#include <type_traits>

static_assert(std::is_same_v<std::remove_reference_t<decltype("foo")>,
const char[4]>);
static_assert(std::ranges::equal("foo", std::array{'f', 'o', 'o', '\0'}));
```

A common workaround is to wrap the string literal in `std::string_view`. But this has a performance cost: despite the fact that we only care about the first five characters, we still need to make a pass through the entire string to find the null terminator:

```
constexpr std::string take_five(char const* long_string) {
    std::string_view const long_string_view = long_string; // read all of
long_string!
    return std::string{std::from_range, long_string_view |
std::views::take(5)};
}
```

This paper introduces `null_sentinel` to solve this problem. It ends the range when it encounters a null:

```
constexpr std::string take_five(char const* long_string) {
    std::ranges::subrange const long_string_range(long_string,
std::null_sentinel); // lazy!
    return std::string{std::from_range, long_string_range |
std::views::take(5)};
}
```

It also introduces a `null_term` CPO for the common case of constructing the a subrange like the one in the above example:

```
constexpr std::string take_five(char const* long_string) {  
    return std::string{std::from_range, std::null_term(long_string) |  
        std::views::take(5)};  
}
```

The sentinel type matches any iterator position it at which `*it` is equal to a default-constructed object of type `iter_value_t<I>`. This works for null-terminated strings, but can also serve as the sentinel for any range terminated by a default-constructed value.

For example, `null_term` can be used to iterate `argv` and `environ`. The following program demonstrates this:

```
#include <print>  
  
extern char** environ;  
  
int main(int argc, char** argv) {  
    std::println("argv: {}", std::null_term(argv));  
    std::println("environ: {}", std::null_term(environ));  
}
```

Output:

```
$ env --ignore-environment F00=bar BAZ=quux ./test corge  
argv: ["/test", "corge"]  
environ: ["F00=bar", "BAZ=quux"]
```

§ 2 Design Alternatives

This paper includes two alternatives for the sentinel: one that always passes the iterator to the equality operator by reference, and one that passes by reference for input iterators and by value otherwise. (Input iterators must be passed by reference here because they are allowed to be noncopyable.)

The argument for the by-reference design is that it's simpler. The argument for the by-value design is that it's more consistent, since, to the author's knowledge, every other parameter in the standard library that takes an iterator takes it by value; although we still need to take input iterators by reference here, we allow the common case to be consistent and treat the edge case as an edge case.

Finally, we could also simply drop `null_sentinel` entirely, only standardize `null_term`, and implement `null_term` in terms of `unchecked_take_before`. Note that at the time of writing, there is no paper proposing `unchecked_take_before`, so that would also need to be written if we want to take that direction.

§ 2.1 Sentinel-based design

§ 2.1.1 Exposition-only helper concept *default-initializable-and-equality-comparable-iter-value*

```
namespace std {  
  
    template<class I>  
    concept default-initializable-and-equality-comparable-iter-value =  
        default_initializable<iter_value_t<I>> &&  
        equality_comparable_with<iter_reference_t<I>, iter_value_t<I>>; //  
        exposition only  
  
}
```

§ 2.1.2 null_sentinel pass-by-value design

§ 2.1.2.1 Null sentinel

```
namespace std {  
  
    struct null_sentinel_t {  
        template<input_iterator I>  
            requires (not forward_iterator<I>) && default-initializable-and-  
                equality-comparable-iter-value<I>  
        friend constexpr bool operator==(I const& it, null_sentinel_t);  
        template<forward_iterator I>  
            requires default-initializable-and-equality-comparable-iter-value<I>  
        friend constexpr bool operator==(I it, null_sentinel_t);  
    };  
  
    inline constexpr null_sentinel_t null_sentinel;  
  
}
```

§ 2.1.2.2 Operations

```
template<input_iterator I>  
    requires (not forward_iterator<I>) && default-initializable-and-  
        equality-comparable-iter-value<I>  
friend constexpr bool operator==(I const& it, null_sentinel_t);
```

Effects:

Equivalent to `return *it == iter_value_t<I>{};`.

```
template<forward_iterator I>  
    requires default-initializable-and-equality-comparable-iter-value<I>  
friend constexpr bool operator==(I it, null_sentinel_t);
```

Effects:

Equivalent to `return *it == iter_value_t<I>{};`.

§ 2.1.3 null_sentinel pass-by-reference design

§ 2.1.3.1 Null sentinel

```
namespace std {  
  
    struct null_sentinel_t {  
        template<input_iterator I>  
            requires default-initializable-and-equality-comparable-iter-value<I>  
            friend constexpr bool operator==(I const& it, null_sentinel_t);  
    };  
  
    inline constexpr null_sentinel_t null_sentinel;  
  
}
```

§ 2.1.3.2 Operations

```
template<input_iterator I>  
    requires default-initializable-and-equality-comparable-iter-value<I>  
    friend constexpr bool operator==(I it, null_sentinel_t);
```

Effects:

Equivalent to `return *it == iter_value_t<I>{};`.

§ 2.1.4 null_term CPO

```
namespace std {  
  
    inline constexpr unspecified null_term;  
  
}
```

The name `null_term` denotes a customization point object ([customization.point.object]). Given a subexpression `E`, the expression `null_term(E)` is expression-equivalent to `ranges::subrange(E, null_sentinel)`.

§ 2.2 unchecked_take_before-based design

§ 2.2.1 exposition-only helper concept *default-initializable-iter-value*

```
namespace std {  
  
    template<class I>  
    concept default-initializable-iter-value =  
        default_initializable<iter_value_t<I>>; // exposition only  
  
}
```

§ 2.2.2 null_term CPO

```
namespace std {
```

```
inline constexpr unspecified null_term;  
}
```

The name `null_term` denotes a customization point object ([`customization.point.object`]). Given a subexpression `E`, the expression `null_term(E)` is expression-equivalent to `subrange(E, unreachable_sentinel)` | `unchecked_take_before(iter_value_t<decltype(E)>{{})`.

Constraints:

default-initializable-iter-value<`E`> is `true`.

§ 3 History

§ 3.1 Changelog

This proposal was originally written by Zach Laine as part of P2728, then updated and split out by Eddie Nolan.

§ 3.1.1 Changes since P2728R1

- Generalize `null_sentinel_t` to a non-Unicode-specific facility.

§ 3.1.2 Changes since P2728R3

- Change `null_sentinel_t` back to being Unicode-specific.

§ 3.1.3 Changes since P2728R4

- Move `null_sentinel_t` to `std`, remove its base member function, and make it useful for more than just pointers, based on SG-9 guidance.

§ 3.1.4 Changes since P2728R5

- Simplify the complicated constraint on the comparison operator for `null_sentinel_t`.

§ 3.1.5 Changes since P2728R6

- Fix a bug in `null_sentinel_t` causing it not to satisfy `sentinel_for` by changing its `operator==` to return `bool`.
- Fix a bug in `null_sentinel_t` where it did not support non-copyable input iterators by having `operator==` take input iterators by reference.

§ 3.1.6 Changes since P2728R7

- Move `null_sentinel` and `null_term` into P3705

§ 3.1.7 Changes since P3705R0

- Fix off-by-one in textual description in motivation section
- Add example with argv and environ
- Add unchecked_take_before design alternative

§ 3.2 Relevant Polls/Minutes

§ 3.2.1 SG9 review of D2728R4 on 2023-06-12 during Varna 2023

— Minutes

POLL: Move null_sentinel_t to std:: namespace

SF	F	N	A	SA
1	3	1	0	0

Of Authors: 1

Author's Position: F

Attendance: 9 (4 abstentions)

Outcome: Consensus in favor

POLL: Remove null_sentinel_t::base member function from the proposal

SF	F	N	A	SA
0	4	1	0	0

Of Authors: 1

Author's Position: F

Attendance: 8 (3 abstentions)

Outcome: Consensus in favor

§ 3.2.2 SG16 review of P2728R3 on 2023-05-10 (Telecon)

— Minutes

POLL: Separate std::null_sentinel_t from P2728 into a separate paper for SG9 and LEWG; SG16 does not need to see it again.

SF	F	N	A	SA
1	1	4	2	1

Attendance: 12 (3 abstentions)

Outcome: No consensus; author's discretion for how to continue.